



**UNIVERSIDAD DE
COSTA RICA**
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 3: Persistencia Avanzada, Relaciones Bidireccionales y Optimización de Consultas en Spring Boot con SQL Server

Metadatos

- **Tiempo Estimado:** 5 horas (trabajo extraclase autónomo)

Herramientas Requeridas:

- Java JDK (instalado localmente).
- Maven 3.8+ o Maven Wrapper.
- Microsoft SQL Server Developer Edition.
- Microsoft SQL Server Management Studio (SSMS).

- Visual Studio Code o IntelliJ IDEA.

Metas de Aprendizaje:

1. Mapear relaciones bidireccionales One-to-Many utilizando anotaciones JPA (`@OneToMany` y `@ManyToOne`), controlando la integridad referencial con `orphanRemoval`.
2. Habilitar e implementar la auditoría automática en tablas físicas mediante la herencia de una súper clase JPA (`@MappedSuperclass`).
3. Aprovechar los estados de persistencia e internals de Hibernate (como *Dirty Checking*) para simplificar las operaciones de actualización.
4. Diagnosticar el problema de rendimiento N+1 SELECT mediante trazas en consola e implementar su solución óptima con consultas `JOIN FETCH`.

Introducción

En aplicaciones empresariales de gran escala, el diseño de la base de datos y la eficiencia de la capa de acceso a datos son pilares indispensables para garantizar la escalabilidad. La incorrecta administración del contexto de persistencia (Caché L1) y la sobrecarga de consultas a base de datos son las causas más comunes de caídas del sistema bajo entornos transaccionales masivos.

En este laboratorio calificado de carácter individual, usted construirá el backend para un **Sistema de Gestión de Tareas de Proyecto (Project Task Manager)** utilizando Spring Boot, Hibernate y Microsoft SQL Server. Los estudiantes aplicarán los conocimientos adquiridos en la **Práctica Guiada 4** para estructurar un modelo de persistencia avanzado, configurar la auditoría de registros de forma automática y optimizar las consultas complejas.

Parte 1: Estructura de Base de Datos (SQL Server)

Usted debe crear y administrar una base de datos local llamada `GestorTareas[Carné]_II2026` utilizando SQL Server Management Studio.

Ejecute el siguiente script para establecer la base de datos inicial (asegúrese de reemplazar [Carné] por su carné en mayúsculas, ej. B98765):

```
CREATE DATABASE GestorTareas[Carné]_II2026;
GO

USE GestorTareas[Carné]_II2026;
GO

-- 1. Tabla Colaborador
CREATE TABLE Colaborador (
    colaborador_id INT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    correo VARCHAR(100) NOT NULL UNIQUE
);

-- 2. Tabla Tarea
CREATE TABLE Tarea (
    tarea_id INT IDENTITY(1,1) PRIMARY KEY,
    titulo VARCHAR(100) NOT NULL,
    descripcion VARCHAR(500) NOT NULL,
    estado VARCHAR(30) NOT NULL,
    colaborador_id INT NOT NULL,
    CONSTRAINT FK_Tarea_Colaborador FOREIGN KEY (colaborador_id)
        REFERENCES Colaborador(colaborador_id)
);

-- 3. Tabla Comentario
CREATE TABLE Comentario (
    comentario_id INT IDENTITY(1,1) PRIMARY KEY,
    texto VARCHAR(500) NOT NULL,
    tarea_id INT NOT NULL,
    fecha_creacion DATETIME NULL,
    fecha_modificacion DATETIME NULL,
    CONSTRAINT FK_Comentario_Tarea FOREIGN KEY (tarea_id)
        REFERENCES Tarea(tarea_id) ON DELETE CASCADE
);

-- 4. Semillas de datos
INSERT INTO Colaborador (nombre, correo)
VALUES ('Juan Pérez', 'juan.perez@ucr.ac.cr'),
       ('María López', 'maria.lopez@ucr.ac.cr');

INSERT INTO Tarea (titulo, descripcion, estado, colaborador_id)
VALUES ('Diseñar BD', 'Crear script de tablas físicas', 'Pendiente', 1),
       ('API REST', 'Implementar controladores Spring', 'En Progreso', 2);
```

Parte 2: Requerimientos del Backend (Spring Boot)

Su proyecto Java debe seguir la arquitectura estándar del curso dividida en capas (`domain`, `data`, `business`, `controller`, `dto`, `config`), reemplazando el paquete `<carnet>` por su carné personal en minúsculas en cada archivo de código fuente.

1. Configuración Maven e Inyección de Dependencias

Configure su `pom.xml` para compilar con la versión de Java que tiene instalada e incluya las dependencias del starter de JPA, el driver JDBC de SQL Server (`mssql-jdbc`) y OpenAPI para la autogeneración de la interfaz Swagger UI.

2. Capa de Mapeo (Domain)

- **Auditoría de Registros:** Cree la súper clase abstracta `AuditableEntity` para capturar automáticamente la fecha de creación y de modificación.
- **Relación Bidireccional:** Mapee la relación entre `Tarea` y `Comentario`. `Tarea` debe poseer una colección de comentarios con borrado físico automático en cascada (`orphanRemoval = true` y `CascadeType.ALL`) configurando adecuadamente el atributo `mappedBy`. La entidad `Comentario` debe heredar de `AuditableEntity`.

3. Capa de Negocio (Services)

Desarrolle el servicio `TareaService` con los siguientes métodos transaccionales: * `registrarTarea(TareaCreationDTO dto)`: Debe validar la existencia del colaborador. Si no existe, retorne una excepción de negocio personalizada (`TareaException`). * `agregarComentario(Integer tareaId, String texto)`: Agrega un comentario a la tarea. Debe comprobar la existencia de la tarea e invocar los métodos auxiliares correspondientes para sincronizar en memoria.

4. Capa de Controladores (API REST)

Exponga los endpoints a través de un controlador REST e implemente documentación interactiva con Swagger UI para: * `POST /api/tareas`: Recibir un JSON para registrar una nueva tarea. * `POST /api/tareas/{id}/comentarios`: Recibir texto plano para agregar un comentario a la tarea especificada.

Parte 3: Internals de Hibernate y Optimización

Como parte crítica de la entrega, usted debe demostrar y documentar que comprende el ciclo de vida de persistencia y la reducción de consultas SQL.

1. Taller de Dirty Checking

En su servicio `TareaService`, implemente el método `completarTarea(Integer tareaId)` que cambie el estado de la tarea a "Completada". Debe recuperar la tarea de la base de datos y realizar el cambio **sin hacer llamadas explícitas a métodos de repositorio como `.save()`**. La transacción debe encargarse de sincronizar el cambio automáticamente en SQL Server.

2. Mitigación del Problema N+1 SELECT

- Habilite en properties la visualización de consultas (`show_sql=true`).
- Cree un endpoint temporal `GET /api/tareas/nplusone` que itere las tareas y sus comentarios correspondientes en un bucle, forzando la carga perezosa (LAZY). Registre en su documentación la traza de consola que muestra las múltiples consultas SELECT.
- Diseñe una consulta optimizada con la cláusula **JOIN FETCH** en `TareaRepository` y exponga el endpoint `GET /api/tareas/optimizada`. Valide que Hibernate ejecute **una única consulta SQL** para obtener la colección completa con sus asociaciones en consola.

Pistas y Ayudas (Hints para el Éxito)

Pista 1: Habilitar el Interceptor de Auditoría

No olvide agregar la anotación `@EnableJpaAuditing` en una clase de configuración (ej: `JpaConfig.java` dentro del paquete `config`). De lo contrario, los campos anotados con `@CreatedDate` y `@LastModifiedDate` quedarán nulos al insertar datos.

Pista 2: Evitar Excepciones de Lazy Loading

Recuerde que cuando las relaciones se configuran como `FetchType.LAZY`, el acceso a la colección de elementos secundarios fuera de una transacción activa lanzará la excepción `LazyInitializationException`. Si necesita devolver las tareas y comentarios en su controlador, utilice la consulta optimizada con `JOIN FETCH` para precargar los datos antes de cerrar el hilo de persistencia.

Pista 3: Limpieza de Caché L1 en Actualizaciones Masivas

Al ejecutar una consulta nativa o JPQL de actualización masiva con `@Modifying`, es altamente recomendable configurar `clearAutomatically = true` para forzar a Hibernate a vaciar los objetos obsoletos que aún residen en la memoria RAM del servidor.

Rúbrica de Evaluación

La nota final de este Laboratorio 3 se evaluará bajo los siguientes porcentajes:

Criterio	Porcentaje	Descripción Detallada
Mapeo y Relación Bidireccional	30%	Mapeo JPA de entidades con llaves primarias Identity y relación <code>@OneToMany</code> usando <code>mappedBy</code> , <code>orphanRemoval</code> y <code>Cascade</code> .
Auditoría Automática	15%	Súper clase <code>@MappedSuperclass</code> , escuchadores de eventos y configuración <code>@EnableJpaAuditing</code> funcionando en SQL Server.

Lógica Transaccional (Capa Business)	20%	Capa de servicio con excepciones personalizadas, validación de colaboradores y actualizaciones automáticas por Dirty Checking.
Optimización de Consultas (N+1)	20%	Endpoint de demostración de error N+1 en consola y método optimizado usando la sintaxis JOIN FETCH en JPA Repository.
Reto Autónomo (Modifying)	15%	Endpoint y repositorio con consulta @Modifying de actualización masiva limpiando correctamente la Caché L1.
Total	100%	Nota final del Laboratorio 3